

ЛР3 (3 пары / 6 часов)

«Изменение статуса и удаление задач. Проверка ввода (Id). Надёжное меню»

Что продолжаем

У вас уже есть:

- `TaskTracker.Core` (модель `TaskItem`, enum `TaskStatus`, `TaskService` с Add/GetAll)
- `TaskTracker.App` (консольное меню)

Теперь добавим:

- **изменение статуса** задачи по Id
 - **удаление** задачи по Id
 - обработку ошибок: "Id не число", "такой задачи нет"
-

0) Что должно быть готово в конце ЛР3

✓ В меню появились пункты:

- 3) Изменить статус задачи
- 1. Удалить задачу

✓ Работают сценарии:

- пользователь вводит Id → статус меняется
- пользователь вводит Id → задача удаляется
- если Id не число → понятная ошибка
- если Id не найден → понятная ошибка

✓ В репозитории есть:

- `docs/LR3_Report.md`

- минимум 3 коммита за ЛРЗ
-

1) Теория (важное, но развернуто)

1.1. Почему Id нужен

Id — это “номер задачи”.

По названию искать сложно (может быть два одинаковых).

Поэтому операции “изменить” и “удалить” делаем по Id.

1.2. Проверка ввода (не доверяем пользователю)

Пользователь может ввести:

- буквы вместо числа (“abc”)
- пусто
- число, которого нет в списке (999)

Правило для начинающих:

Наша программа **не падает** и **пишет понятное сообщение**, что делать.

1.3. Исключения (очень просто)

В Core мы можем “сказать”, что что-то пошло не так, через `throw`.

В App мы “ловим” ошибку и показываем сообщение.

Это удобно:

Core отвечает за “правила”, App отвечает за “общение с человеком”.

1.4. Перечисление (enum) для статусов

Статус — это “одно из трёх”:

- New
- InProgress

- Done

Enum гарантирует, что не будет “левых” значений.

2) Практика. Пара 1: добавляем методы UpdateStatus и Delete в TaskService

Шаг 1. Открыть решение

1. Visual Studio → открыть `TaskTracker.sln`
 2. Убедиться, что проект собирается (Build → Build Solution)
-

Шаг 2. Открыть TaskService

1. В Solution Explorer: `TaskTracker.Core → Services → TaskService.cs`
 2. Откройте файл
-

Шаг 3. Добавить метод “найти задачу по Id”

Внутри класса `TaskService` добавьте метод:

```
private TaskItem GetExisting(int id)
{
    var task = _tasks.FirstOrDefault(t => t.Id == id);
    if (task is null)
        throw new ArgumentException($"Задача с Id={id} не найдена.");
    return task;
}
```

Если VS подсвечивает FirstOrDefault красным — сверху файла добавьте:

```
using System.Linq;
```

Шаг 4. Добавить метод ChangeStatus

Ниже добавьте:

```
public TaskItem ChangeStatus(int id, TaskStatus newStatus)
{
    var task = GetExisting(id);
    task.Status = newStatus;
    return task;
}
```

Шаг 5. Добавить метод Delete

Ниже добавьте:

```
public void Delete(int id)
{
    var task = GetExisting(id);
    _tasks.Remove(task);
}
```

Шаг 6. Сборка

1. Build → Build Solution
2. Должно быть `Build succeeded`

Шаг 7. Коммит №1 (Core: статус и удаление)

Git Changes → сообщение:

- `feat: add change status and delete methods to TaskService`

Commit All → Push.

3) Практика. Пара 2: обновляем меню в Program.cs (пункты 3 и 4)

Шаг 8. Открыть Program.cs

1. `TaskTracker.App → Program.cs`
-

Шаг 9. Добавить пункты меню

В вашем меню добавьте строки:

- `3) Изменить статус задачи`
- `4) Удалить задачу`

(оставьте `1,2,0` как было)

Шаг 10. Сделать вспомогательную функцию чтения числа (Id)

Очень важно для новичков: чтобы не копировать `TryParse` 100 раз.

В `Program.cs` (ниже `var service = new TaskService();`, но до `while(true)`) вставьте:

```
static bool TryReadInt(string prompt, out int value)
{
    Console.Write(prompt);
    var text = Console.ReadLine();
    return int.TryParse(text, out value);
}
```

Шаг 11. Реализовать пункт 3: изменить статус

Внутри `while(true)` добавьте обработку `input == "3"`:

```
if (input == "3")
{
```

```

var tasks = service.GetAll();
if (tasks.Count == 0)
{
    Console.WriteLine("Список задач пуст. Нечего менять.");
    continue;
}

Console.WriteLine("Список задач:");
foreach (var t in tasks)
    Console.WriteLine($"{t.Id}. {t.Title} [{t.Status}]");

if (!TryReadInt("Введите Id задачи: ", out var id))
{
    Console.WriteLine("Ошибка: Id должно быть числом.");
    continue;
}

Console.WriteLine("Выберите новый статус:");
Console.WriteLine("0 - New (Новая)");
Console.WriteLine("1 - InProgress (В работе)");
Console.WriteLine("2 - Done (Готово)");

if (!TryReadInt("Введите статус (0/1/2): ", out var statusNumber))
{
    Console.WriteLine("Ошибка: статус должен быть числом 0/1/2.");
    continue;
}

if (statusNumber < 0 || statusNumber > 2)
{
    Console.WriteLine("Ошибка: статус должен быть 0, 1 или 2.");
    continue;
}

```

```

    }

    var newStatus = (TaskStatus)statusNumber;

    try
    {
        var updated = service.ChangeStatus(id, newStatus);
        Console.WriteLine($"Статус изменён: #{updated.Id} {updated.Title} [{updated.Status}]");
    }
    catch (ArgumentException ex)
    {
        Console.WriteLine("Ошибка: " + ex.Message);
    }

    continue;
}

```

Шаг 12. Реализовать пункт 4: удалить задачу

Добавьте обработку `input == "4"`:

```

if (input == "4")
{
    var tasks = service.GetAll();
    if (tasks.Count == 0)
    {
        Console.WriteLine("Список задач пуст. Нечего удалять.");
        continue;
    }

    Console.WriteLine("Список задач:");
    foreach (var t in tasks)
        Console.WriteLine($"{t.Id}. {t.Title} [{t.Status}]");
}

```

```

        if (!TryReadInt("Введите Id задачи для удаления: ", out v
ar id))
        {
            Console.WriteLine("Ошибка: Id должно быть числом.");
            continue;
        }

        Console.Write("Точно удалить? (y/n): ");
        var answer = (Console.ReadLine() ?? "").Trim().ToLower();

        if (answer != "y")
        {
            Console.WriteLine("Удаление отменено.");
            continue;
        }

        try
        {
            service.Delete(id);
            Console.WriteLine($"Задача с Id={id} удалена.");
        }
        catch (ArgumentException ex)
        {
            Console.WriteLine("Ошибка: " + ex.Message);
        }

        continue;
    }
}

```

Шаг 13. Проверка (F5) — обязательный сценарий

Запустите программу (F5) и проверьте:

Сценарий 1: нормальная работа

1. Добавьте 2 задачи (пункт 1)
2. Показать список (2)
3. Изменить статус задачи (3):
 - введите Id 1
 - выберите статус 2
4. Показать список (2) — статус должен измениться
5. Удалить задачу (4):
 - Id 2
 - у
6. Показать список — осталась 1 задача

Сценарий 2: ошибки

1. Пункт 3 → Id = `abc` → должно написать "Id должно быть числом"
 2. Пункт 4 → Id = `999` → должно написать "не найдена"
 3. Пункт 3 → статус = `5` → "статус должен быть 0/1/2"
-

Шаг 14. Коммит №2 (App: статус/удаление)

Git Changes → сообщение:

- `feat: add change status and delete commands in console app`

Commit All → Push.

4) Практика. Пара 3: отчёт + "полировка" + чистые сообщения

Шаг 15. Добавить отчёт `docs/LR3_Report.md`

Создайте файл `docs/LR3_Report.md` и вставьте:

Отчёт ЛР3 – TaskTracker

Что сделано

- В TaskService добавлены методы:
 - ChangeStatus(id, status)
 - Delete(id)
- В консольном меню добавлены пункты:
 - 3) Изменить статус задачи
 - 4) Удалить задачу
- Добавлена проверка ввода Id и статуса (TryParse и диапазон)

Как проверить

- 1) Запуск (F5)
- 2) Добавить 2 задачи
- 3) Изменить статус задачи Id=1 на Done
- 4) Удалить задачу Id=2 (подтверждение y)
- 5) Проверить список
- 6) Проверить ошибки:
 - Id = abc
 - Id = 999
 - Status = 5

Шаг 16. “Полировка” для слабой группы (очень полезно)

Добавьте в пункт “Показать список” (2) вывод подсказки:

- “Нажмите 3, чтобы изменить статус, и 4, чтобы удалить”.

Можно просто строкой:

```
Console.WriteLine("Подсказка: пункт 3 – изменить статус, пункт 4 – удалить.");
```

Шаг 17. Коммит №3 (отчёт + мелкие улучшения)

Сообщение:

- `docs: add LR3 report and small UX improvements`

Commit All → Push.

5) Что сдавать (чек-лист)

Студент сдаёт:

1. ☒ ссылка на GitHub репозиторий
 2. ☒ скрин/видео 30–60 сек:
 - добавление 2 задач
 - изменение статуса
 - удаление
 3. ☒ `docs/LR3_Report.md`
 4. ☒ минимум 3 коммита за ЛР3
-

6) Оценивание (просто)

“3”

- работает добавление + список + изменение статуса (без удаления тоже можно, но лучше с ним)
- есть отчёт

“4”

- работает полный набор (add/list/status/delete)
- обработка ошибок Id и статуса

“5”

- очень аккуратные сообщения и сценарии

- “защита от дурака” везде (пустые списки, неверные вводы)
 - чистая история коммитов
-